



Pflichtenheft

Projektname	Getränkespender
Projektnummer	
Projektleiter	Raphael Thaler
Erstellt am	09.01.2026
Letzte Änderung am	16.01.2026
Status	In Bearbeitung
Aktuelle Version	1.1

Änderungsverlauf

Nr.	Datum	Version	Geänderte Kapitel	Art der Änderung	Autor
1	09.01.2026	1.0	Alle	Erstellung	Luca Aigner
2	16.01.2026	1.1	5.3, 5.4, 6.2	Erweiterung	Luca Aigner
3	27.01.2026	1.1	4.4, 5.3.2	Erweiterung/Änderung	Luca Aigner



Inhalt

1	Einleitung	4
2	Allgemeines	4
2.1	Ziel und Zweck des Dokuments	4
2.2	Ausgangssituation	4
2.3	Projektbezug	4
2.4	Abkürzungen	4
2.5	Teams und Schnittstellen	4
3	Konzept	5
3.1	Ziel(e) des Anbieters	5
3.2	Ziel(e) und Nutzen des Anwenders	5
3.3	Zielgruppe(n)	5
3.3.1	Endanwender (Konsumenten / Partygäste) Diese Gruppe nutzt das Gerät ausschließlich zur Getränkeentnahme.	5
3.3.2	Betreiber (Gastgeber / Administratoren) Diese Gruppe ist für die Inbetriebnahme, Befüllung und Konfiguration der Mischverhältnisse zuständig.	6
3.3.3	Privatanwender (Haushalt / Eigenbedarf) Diese Zielgruppe stellt eine Personalunion aus Betreiber und Endanwender dar. Das Gerät wird im privaten Wohnbereich dauerhaft oder temporär aufgestellt.	6
4	Funktionale Anforderungen	6
4.1	Kernfunktionalitäten (Hardware & Steuerung)	6
4.2	Web-Interface & Konfiguration (React App)	7
4.3	Datenhaltung & Kommunikation	7
4.4	Optionale Erweiterung: Gamification-Modul (Stochastisches Belohnungssystem) Dieses Modul dient der Erhöhung der Benutzerinteraktion durch einen spielerischen Ansatz ("Slot-Machine-Effekt").	7
5	Nichtfunktionale Anforderungen	8
5.1	Allgemeine Anforderungen	8
5.2	Gesetzliche Anforderungen	9
5.3	Technische Anforderungen	9
5.3.1	Web:	9
5.3.2	Hardware:	10
5.4	Betriebliche Anforderungen	11
6	Rahmenbedingungen	12
6.1	Zeitplan	12
6.2	Rahmenbedingungen: Technische Anforderungen (Entwicklungsumgebung)	12



6.3	Problemanalyse.....	13
6.4	Qualität.....	14
7	Liefer- und Abnahmebedingungen.....	14
8	Anhang.....	14



1 Einleitung

Das vorliegende Pflichtenheft enthält die an das zu entwickelnde Produkt gestellten funktionalen sowie nicht-funktionalen Anforderungen. Es dient als Basis für die Ausschreibung und Vertragsgestaltung und bildet somit die Vorgabe für die Angebotserstellung. Kommt es zwischen Auftragnehmer und Auftraggeber zu einem Vertragsabschluss, ist das bestehende Pflichtenheft rechtlich bindend. Alle zuvor zwischen Auftraggeber und Auftragnehmer getroffenen Absprachen verlieren in der Regel durch das Pflichtenheft ihre Gültigkeit – sofern hier nichts Gegenteiliges vermerkt ist. Mit den Anforderungen werden die Rahmenbedingungen für die Entwicklung festgelegt, die vom Auftragnehmer im Pflichtenheft detailliert ausgestaltet werden.

2 Allgemeines

2.1 Ziel und Zweck des Dokuments

Dieses Pflichtenheft beschreibt ein Projekt über einen Getränkenspender.

2.2 Ausgangssituation

Dieses Projekt wird in schulischen Verhältnissen durchgeführt von Raphael Thaler, Luca Aigner, Lukas Gutmann und Clemens Deutinger.

2.3 Projektbezug

Das vorliegende Projekt ist ein unabhängiges Projekt.

2.4 Abkürzungen

CT01 ist CodingTeam01

DT01 ist DesignTeam01

HWT01 ist HardwareTeam01

2.5 Teams und Schnittstellen

Rolle(n)	Name	Telefon	E-Mail	Team
Projektleiter	Raphael Thaler	9116967	Raphael.thaler@htl-saalfelden.at	CT01
Entwickler	Luca Aigner	06601442142	Luca.aigner@htl-saalfelden.at	CT01
3D-Designer	Clemens Deutinger	06601411251	Clemens.deutinger@htl-saalfelden.at	DT01
Systemelektroniker	Lukas Gutmann	066704043305	Lukas.gutmann@htl-saalfelden.at	HWT01

Die Kommunikation erfolgt größtenteils von Person zu Person aber auch über Teams. Enge Zusammenarbeit besteht zwischen dem DT01 und HWT01 aber auch zwischen Kommunikationen mit CT01.



3 Konzept

3.1 Ziel(e) des Anbieters

Nachweis technischer Kompetenz (Proof of Concept): Es soll demonstriert werden, dass eine nahtlose Integration von Embedded Systems (ESP32/MicroPython) und modernen Web-Technologien (React) realisierbar ist. Ziel ist die Schaffung einer stabilen Kommunikationsschnittstelle (API) zwischen der Hardware-Steuerung und der Benutzeroberfläche.

Erstellung eines funktionsfähigen Prototyps (MVP): Ziel ist die Fertigstellung eines "Minimum Viable Product", das die Kernfunktionen (Pumpensteuerung, Konfiguration, Webserver) zuverlässig und reproduzierbar erfüllt. Der Prototyp soll als Basis für potenzielle spätere Erweiterungen (z.B. mehr Pumpen, Cloud-Anbindung) dienen.

Gewährleistung der Betriebssicherheit: Durch den Einsatz geeigneter Hardware-Komponenten (Gehäuse, MOSFET-Treiberstufen) und Software-Routinen soll ein sicherer Betrieb gewährleistet werden, bei dem Elektronik und Flüssigkeiten strikt getrennt sind, um Kurzschlüsse oder Fehlfunktionen zu vermeiden.

Benutzerfreundlichkeit (Usability) ohne Vorkenntnisse: Die einfache Bedienung mittels Knöpfe und eines Gefäßes. Die Funktion der Knöpfe werden mittels einem Papierlabel beschrieben.

3.2 Ziel(e) und Nutzen des Anwenders

Gemütliche Zubereitung eines Mischgetränks durch einen Knopfdruck.

3.3 Zielgruppe(n)

3.3.1 Endanwender (Konsumenten / Partygäste) Diese Gruppe nutzt das Gerät ausschließlich zur Getränkeentnahme.

Charakteristika: Technisch oft Laien; befinden sich in einem geselligen Umfeld (Party, Event); erwarten sofortige Ergebnisse ohne Einarbeitungszeit.

Anforderungen an das Produkt: Die Bedienung muss extrem simpel und selbsterklärend sein ("Walk-up-and-use"). Da diese Nutzer nur mit der Hardware (Gehäuse/Taster) interagieren, stehen Robustheit und klare Beschriftung (Papierschilder) im Vordergrund. Fehleingaben oder wildes Drücken der Tasten müssen softwareseitig abgefangen werden.



3.3.2 Betreiber (Gastgeber / Administratoren) Diese Gruppe ist für die Inbetriebnahme, Befüllung und Konfiguration der Mischverhältnisse zuständig.

Charakteristika: Besitzen grundlegende Kenntnisse im Umgang mit Smartphones oder Laptops; sind verantwortlich für den reibungslosen Ablauf.

Anforderungen an das Produkt: Sie benötigen Zugriff auf die Konfigurationsebene (React-Webinterface). Für sie muss die Oberfläche übersichtlich gestaltet sein, um schnell Rezepte ändern zu können, ohne den Programmcode (MicroPython) verstehen oder bearbeiten zu müssen. Eine mobile Optimierung der Webseite ist für diese Gruppe essenziell, da die Konfiguration vermutlich spontan über das Smartphone erfolgt.

3.3.3 Privatanwender (Haushalt / Eigenbedarf) Diese Zielgruppe stellt eine Personalunion aus Betreiber und Endanwender dar. Das Gerät wird im privaten Wohnbereich dauerhaft oder temporär aufgestellt.

Charakteristika: Der Nutzer konfiguriert das Gerät nach seinen persönlichen Vorlieben und nutzt es anschließend selbst für den täglichen Bedarf oder im familiären Rahmen. Die Frequenz der Nutzung ist geringer als im Party-Betrieb, dafür erfolgt der Einsatz über einen längeren Zeitraum.

Anforderungen an das Produkt: Hier stehen **Komfort** und **Wartungsfreundlichkeit** im Fokus. Da der Nutzer auch für die Reinigung zuständig ist, muss der Prozess des Durchspülens oder Wechselns der Flüssigkeiten möglichst einfach gestaltet sein. Zudem spielen Ästhetik und Platzbedarf eine größere Rolle, da das Gerät in den Wohnraum integriert wird. Die Geräuscentwicklung der Pumpen sollte auf ein wohnraumtaugliches Maß beschränkt sein.

4 Funktionale Anforderungen

4.1 Kernfunktionalitäten (Hardware & Steuerung)

- **/F010/ Zeitgesteuerte Pumpenansteuerung:** Das System muss in der Lage sein, zwei Gleichstrompumpen (Pumpe A und Pumpe B) unabhängig voneinander ein- und auszuschalten. Die Ansteuerung erfolgt zeitbasiert (Dauer in Sekunden/Millisekunden).
- **/F020/ Mischprofile abrufen:** Das System muss auf dem Mikrocontroller (ESP32) drei vordefinierte Profile ("Rezepte") vorhalten. Ein Profil besteht aus einem Datensatz, der die Laufzeit für Pumpe A und Pumpe B definiert.
- **/F030/ Physische Auslösung (Trigger):** Über drei Hardware-Taster am Gehäuse muss der Mischvorgang gestartet werden können.
 - Taster 1 löst Profil 1 aus.
 - Taster 2 löst Profil 2 aus.
 - Taster 3 löst Profil 3 aus.



- **/F040/ Blockierfunktion (Interlock):** Während ein Pumpvorgang läuft, müssen Eingaben über die anderen Taster ignoriert werden, um eine Doppel-Dosierung oder Fehler im Ablauf zu verhindern.
- **/F050/ Not-Abbruch:** Es muss eine Möglichkeit geben, einen laufenden Pumpvorgang sofort zu beenden (z.B. durch langes Drücken eines beliebigen Tasters oder einen dedizierten Reset), falls ein Glas überläuft.

4.2 Web-Interface & Konfiguration (React App)

- **/F060/ Dashboard-Ansicht:** Die Web-Applikation muss den aktuellen Status des Systems anzeigen (z.B. "Bereit", "Mischt gerade", "Fehler").
- **/F070/ Rezept-Konfiguration:** Die Webseite muss ein Formular bereitstellen, mit dem die Laufzeiten für Pumpe A und Pumpe B für jeden der drei Taster (Slots) individuell eingestellt werden können.
 - Eingabeformat: Numerisch (z.B. in Sekunden).
- **/F080/ Validierung:** Die Eingaben müssen vor dem Senden validiert werden (z.B. keine negativen Zeiten, maximale Laufzeitbegrenzung zum Schutz der Pumpen).
- **/F090/ Manuelle Steuerung (Wartungsmodus):** Über das Web-Interface sollen die Pumpen manuell gestartet und gestoppt werden können (z.B. zum Ansaugen der Flüssigkeit nach Neubefüllung oder zur Reinigung), ohne einen automatischen Timer zu nutzen.

4.3 Datenhaltung & Kommunikation

- **/F100/ Persistente Speicherung:** Änderungen an den Rezepten, die über das Web-Interface vorgenommen werden, müssen dauerhaft im Flash-Speicher des ESP32 gespeichert werden (z.B. als JSON-Datei). Die Einstellungen müssen nach einem Stromausfall wiederhergestellt werden.
- **/F110/ API-Schnittstelle:** Der ESP32 muss eine REST-API (oder vergleichbare Schnittstelle) bereitstellen, über die das React-Frontend die aktuellen Konfigurationen lesen (GET) und neue Konfigurationen schreiben (POST) kann.

4.4 Optionale Erweiterung: Gamification-Modul (Stochastisches Belohnungssystem)

Dieses Modul dient der Erhöhung der Benutzerinteraktion durch einen spielerischen Ansatz ("Slot-Machine-Effekt").

- **/F_OPT_010/ Signalverarbeitung Münzeinwurf:** Das System muss digitale Impulse eines externen Münzprüfers (Coin Acceptor) erfassen. Ein valider Einwurf wird als "Trigger-Event" gewertet, der das System aus dem Ruhezustand weckt.
- **/F_OPT_020/ Physische Interaktion (Hebel/Trigger):** Der Spielmechanismus wird über einen physischen Aktor (Hebelmechanik mit Mikroschalter oder Arcade-Button) gestartet.
 - Sperre: Der Hebel ist standardmäßig softwareseitig deaktiviert. Betätigungen ohne vorherigen Münzeinwurf werden ignoriert (ggf. mit visueller Fehlermeldung).
 - Freigabe: Erst nach registriertem Münzeinwurf wird der Hebel für genau einen Spielzyklus freigegeben.



- /F_OPT_030/ Lokale Visualisierung (Display): Das Gerät verfügt über ein integriertes Display zur direkten Nutzerführung am Gerät, unabhängig vom Web-Interface.
 - Idle-State: Anzeige einer Aufforderung (z.B. "Münze einwerfen").
 - Game-State: Nach Betätigung des Hebels visualisiert das Display den Zufallsprozess (z.B. durchlaufende Symbole).
 - Result-State: Anzeige von "Gewinn" (Auslösen der Pumpen) oder "Niete".
- /F_OPT_040/ Zufallsalgorithmus (RNG): Der Mikrocontroller führt eine Zufallsberechnung durch. Die Gewinnwahrscheinlichkeit (z.B. 1:10) ist variabel.
- /F_OPT_050/ Admin-Konfiguration (Web): Die Konfiguration der Gewinnwahrscheinlichkeit und des ausgegebenen "Gewinn-Getränks" erfolgt ausschließlich über einen geschützten Bereich im React-Webinterface (Admin-Panel), auf den normale Endanwender keinen Zugriff haben.

5 Nichtfunktionale Anforderungen

5.1 Allgemeine Anforderungen

/NF010/ Leistung & Reaktionszeit (Latenz) Da das System physische Flüssigkeiten bewegt, ist die wahrgenommene Reaktionszeit entscheidend.

- **Pumpenstart:** Zwischen dem Drücken eines physischen Tasters und dem Anlaufen der Pumpen darf maximal eine Verzögerung von **3 Sekunden** liegen.
- **Web-Interface:** Da der ESP32 als Webserver begrenzte Ressourcen hat, wird eine Ladezeit der Konfigurations-Webseite von unter **8 Sekunden** angestrebt. Speicher-Operationen (Speichern der Config) müssen innerhalb von **5 Sekunde** bestätigt werden.

/NF020/ Sicherheit (Safety & Security)

- **Elektrische Sicherheit:** Da Elektronik und Flüssigkeiten (leitfähig) in einem Gerät verbaut sind, müssen die flüssigkeitsführenden Teile (Schläuche/Pumpen) physisch von der Steuerelektronik (ESP32/PCB) getrennt sein (z.B. durch eine Trennwand im 3D-Druck-Gehäuse).
- **Eingabevalidierung:** Die Software muss verhindern, dass fehlerhafte Eingaben (z.B. negative Zahlen oder Text statt Zahlen) im Web-Interface zu einem Absturz des Mikrocontrollers führen.
- **Spannung:** Das System arbeitet im für den Menschen ungefährlichen Kleinspannungsbereich (max. 12V DC).

/NF030/ Portabilität & Kompatibilität (Web)

- **Plattformunabhängigkeit:** Das Web-Interface (React) muss auf den gängigen Browsern (Chrome, Safari, Firefox) lauffähig sein.



- **Responsive Design:** Da die Konfiguration vorwiegend "nebenbei" erfolgt, muss das Interface zwingend für **mobile Endgeräte (Smartphones)** optimiert sein (Touch-Bedienung, skalierbare Layouts).

/NF040/ Verfügbarkeit & Robustheit

- **Wi-Fi Reconnect:** Bei Verlust der WLAN-Verbindung muss der ESP32 versuchen, die Verbindung selbstständig wiederherzustellen, ohne dass ein manueller Neustart erforderlich ist.
- **Stand-Alone Betrieb:** Die Kernfunktion (Getränkeausgabe über Taster) muss auch dann funktionieren, wenn keine WLAN-Verbindung besteht oder das Web-Interface gerade nicht aufgerufen wird.

/NF050/ Wartbarkeit & Modularität

- **Code-Trennung:** Die Software-Architektur muss eine klare Trennung zwischen Backend (MicroPython Steuerlogik) und Frontend (React UI) aufweisen.
- **Austauschbarkeit:** Das Gehäuse-Design muss so gestaltet sein, dass Verschleißteile (Schläuche, Pumpenköpfe) zu Reinigungszwecken ohne Zerstörung des Gehäuses entnommen werden können.

5.2 Gesetzliche Anforderungen

Compliance & Sicherheit (Lebensmittel & Elektrik)

- **Lebensmittelkonformität (LFGB / EU 1935/2004):** Alle Bauteile, die in direkten Kontakt mit den Getränken kommen (Schläuche, Pumpenmechanik, Düsen), müssen als "lebensmittelecht" (food grade) zertifiziert sein oder aus entsprechenden Materialien (z.B. Silikon, PETG, Edelstahl) bestehen.

5.3 Technische Anforderungen

5.3.1 Web:

Zielformat (Client-Seite) Da die Benutzeroberfläche als Web-Applikation realisiert wird, ist sie plattformunabhängig konzipiert. Die Unterstützung folgender Umgebungen muss gewährleistet sein:

- Betriebssysteme: Unabhängig (Windows, macOS, iOS, Android, Linux).
- Browser: Die Anwendung muss auf allen modernen, HTML5-fähigen Browsern lauffähig sein (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge). Die Unterstützung des veralteten Internet Explorers ist nicht vorgesehen.
- Mobile Endgeräte: Prioritäre Unterstützung von Smartphones und Tablets (iOS und Android) im Hochformat (Portrait-Mode), da dies das primäre Konfigurationswerkzeug für den "Betreiber" darstellt.

Designvorgaben & User Interface (UI)

- Mobile First & Responsive Design: Die React-Oberfläche muss sich dynamisch an die Bildschirmgröße anpassen. Auf kleinen Bildschirmen (Smartphones) werden Elemente untereinander angeordnet (Stacking), auf größeren Bildschirmen (Laptops) nebeneinander.
- Touch-Optimierung: Alle interaktiven Elemente (Buttons, Slider, Eingabefelder) müssen groß genug dimensioniert sein, um fehlerfrei mit dem Finger bedient werden zu können (Mindestgröße von Touch-Targets ca. 44x44 Pixel).



- **Corporate Design / Styling:** Das Design soll modern und minimalistisch gehalten werden. Es wird ein dunkles Farbschema ("Dark Mode") bevorzugt, da das Gerät voraussichtlich in abgedunkelten Umgebungen (Partys) eingesetzt wird und dies die Blendwirkung reduziert.
Barrierefreiheit (Accessibility) Die Software orientiert sich an grundlegenden Prinzipien der Barrierefreiheit, um die Bedienung auch unter erschwerten Bedingungen (schlechte Beleuchtung, eingeschränkte Motorik) zu ermöglichen:
- **Skalierbarkeit:** Texte müssen über die Browser-Einstellungen vergrößerbar sein, ohne das Layout zu zerstören.
- **Kontraste:** Es ist auf einen ausreichenden Kontrast zwischen Text und Hintergrund zu achten, um die Lesbarkeit in dunklen Räumen zu gewährleisten.
- **Visuelles Feedback:** Interaktionen (z.B. Speichern der Konfiguration) müssen durch eindeutige visuelle Rückmeldungen (Ladebalken, Farbwechsel, "Toast"-Nachrichten) bestätigt werden, damit der Nutzer nicht auf Audio-Feedback angewiesen ist.

5.3.2 Hardware:

Recheneinheit (Mikrocontroller)

- **Komponente:** ESP32 SoC (System on a Chip).
- **Anforderung:** Der Controller muss über integriertes WLAN (2.4 GHz) verfügen, um den Webserver bereitstellen zu können.
- **Speicher:** Ausreichend Flash-Speicher (min. 4MB) für die MicroPython-Firmware, die React-Web-Applikation und die Konfigurationsdateien.
- **I/O-Schnittstellen:** Es werden mindestens 5 GPIOs benötigt (2x Output für Pumpen, 3x Input für Taster).

Aktorik & Fluidik

- **Pumpen:** Einsatz von zwei selbstansaugenden Membranpumpen (6V-12V).
 - *Anforderung:* Die Pumpen müssen lebensmittelecht sein und eine Saughöhe aufweisen, die es erlaubt, Flüssigkeiten aus unterhalb stehenden Behältern zu fördern (Self-priming).
- **Leistungselektronik (Treiberstufe):**
 - Da der Mikrocontroller (3.3V) die Pumpen (6V) nicht direkt treiben kann, ist eine Treiberstufe erforderlich.
 - *Anforderung:* Einsatz von MOSFETs (z.B. IRLZ44N oder Module), um die galvanische Trennung und Leistungsverstärkung sicherzustellen.

Stromversorgung

- **Spannungsquelle:** Externes Netzteil (9V DC), welches genügend Strom (min. 2A) liefert, um beide Pumpen simultan sowie den Controller zu versorgen.
- **Spannungswandlung:** Integration eines Step-Down-Converters (Buck Converter) zur effizienten Wandlung von 9V auf 5V/3.3V für den ESP32.
- **Spannungswandlung (Peripherie):** Falls Komponenten mit 12V-Bedarf (z.B. Standard-Münzprüfer) eingesetzt werden, ist der Einsatz eines Step-Up-Converters (Boost Converter) von 9V auf 12V vorzusehen.

Gehäuse & Physischer Aufbau

- **Material:** Hybrid-Konstruktion aus 3D-Druck-Komponenten (für Halterungen und Passgenauigkeit) und Holz (für die Struktur/Optik).



- **Zwei-Kammer-System:** Konstruktive Trennung in einen "Nassbereich" (Schläuche, Auslassdüse) und einen "Trockenbereich" (ESP32, Netzteil, MOSFETs), um die Elektronik vor Leckagen zu schützen.

Zusatz-Peripherie (Gamification)

- **Display-Einheit:** OLED (z.B. SH1106/SSD1306) oder TFT Modul zur Statusanzeige direkt am Gehäuse.
- **Eingabe-Sensorik:** Mikroschalter (Microswitch) zur Erfassung der Hebel-Position (mechanischer Trigger).
- **Münzverarbeitung:** Elektronischer Münzprüfer mit 12V Puls-Signal.
 - **Interface:** Anbindung an den ESP32 über Pegelwandler oder Spannungsteiler (Level Shifter), um das 12V Signal auf verträgliche 3.3V zu skalieren.

5.4 Betriebliche Anforderungen

Deployment-Prozess (Software-Verteilung) Da das System lokal und ohne Internetanbindung arbeitet, erfolgt das Deployment neuer Software-Versionen nicht über eine Cloud-Pipeline (CI/CD), sondern physisch am Gerät.

- **Firmware-Flash:** Das Aufspielen der MicroPython-Firmware und des React-Frontends (Build-Dateien) erfolgt über eine kabelgebundene USB-Verbindung (Serielle Schnittstelle) mittels geeigneter Tools (z.B. Thonny IDE, esptool).
- **Build-Pipeline:** Die React-Applikation muss vor dem Deployment lokal auf einem Entwickler-PC kompiliert ("build") und minifiziert werden, um den begrenzten Speicherplatz des ESP32 nicht zu überlasten.

Hosting und Infrastruktur (On-Device Hosting)

- **Self-Hosted:** Das System agiert vollständig autark ("On-Premises" / "On-Device"). Der ESP32 übernimmt die Rolle des Webserver und des Datenbank-Servers (Dateisystem). Es ist keine externe Server-Infrastruktur oder Cloud-Anbindung (AWS/Azure) erforderlich.
- **Netzwerk-Infrastruktur:** Das Gerät spannt entweder einen eigenen WLAN-Access-Point (AP-Mode) auf oder integriert sich in ein bestehendes lokales WLAN (Station-Mode).

Backup und Wiederherstellung (Disaster Recovery)

- **Konfigurations-Sicherheit:** Die vom Benutzer erstellten Rezepte (in der config.json) verbleiben im Flash-Speicher. Es ist kein automatisches Cloud-Backup vorgesehen.
- **Factory Reset:** Für den Fall einer fehlerhaften Konfiguration (z.B. WLAN-Passwort falsch eingegeben oder Endlosschleife im Code) muss eine Möglichkeit bestehen, das Gerät auf die Werkseinstellungen zurückzusetzen (z.B. durch Neu-Flashen via USB).
- **Quellcode-Sicherung:** Der Entwicklungsstand (Source Code) wird über ein Versionsverwaltungssystem (z.B. Git/GitHub) gesichert, um bei Datenverlust auf dem Entwickler-PC den Code wiederherstellen zu können.

**Wartung und Pflege (Maintenance)**

- **Software-Wartung:** Fehlerbehebungen (Bugfixes) oder Funktionserweiterungen erfordern den physischen Zugriff auf das Gerät (USB-Anschluss).
- **Hardware-Wartung (Hygiene):** Aufgrund des Lebensmittelkontakts müssen regelmäßige Wartungsintervalle für die Reinigung definiert werden (Durchspülen des Systems mit Wasser/Reinigungsmittel). Das System muss hierfür einen softwaregestützten Wartungsmodus ("Reinigungszyklus") bereitstellen, der beide Pumpen dauerhaft aktiviert.

6 Rahmenbedingungen

Hier gehen Sie zum Beispiel auf die gesamte Bearbeitungszeit ein. Beschreiben Sie ruhig auch die geplanten Betriebs- und Arbeitszeiten.

Wird das Projekt nach einem bestimmten Entwicklungsprozess durchgeführt? (z.B.: V-modell, Wasserfallmodell, Agile, ...)

6.1 Zeitplan

Wie viel Zeit wird für einzelne Phasen voraussichtlich aufgewendet? Hier sollte eine Übersicht folgen, die möglichst auch Arbeitszeiten oder ggf. Betriebspausen miteinbezieht.

6.2 Rahmenbedingungen: Technische Anforderungen (Entwicklungsumgebung)

Für die Realisierung des Projekts werden folgende Technologien, Software-Stacks und Entwicklungswerkzeuge eingesetzt:

Programmiersprachen und Technologien

- **Frontend (Benutzeroberfläche):**
 - **Framework:** React.js (JavaScript/JSX) zur Erstellung einer dynamischen Single-Page-Application (SPA).
 - **Styling:** CSS3 (ggf. mit CSS-Frameworks oder Modulen) für das responsive Design.
 - **Build-Prozess:** Nutzung von Node.js und npm (Node Package Manager), um den React-Code in statische HTML/JS-Dateien zu kompilieren, die auf dem Mikrocontroller gehostet werden können.
- **Backend / Firmware (Steuerung):**
 - **Sprache:** MicroPython. Diese Sprache wurde gewählt, da sie eine schnelle Iteration ermöglicht und leistungsfähige Bibliotheken für die Hardware-Ansteuerung (GPIO, WLAN) sowie einfache Webserver-Funktionalitäten bietet.



- **Kommunikation:** Datenaustausch zwischen Frontend und Backend erfolgt über **HTTP-Requests (REST-API)** im JSON-Format.

Datenbankanforderungen (Datenhaltung)

- **File-Based Storage:** Aufgrund der Ressourcenbeschränkung des ESP32 wird keine relationale Datenbank (wie MySQL) eingesetzt.
- **Format:** Die Speicherung der Konfigurationen (Rezepte, Pumpenlaufzeiten) erfolgt in strukturierten **JSON-Dateien** (config.json) direkt im nicht-flüchtigen Flash-Dateisystem (LittleFS/FAT) des ESP32.
- **Anforderung:** Das Dateisystem muss so implementiert sein, dass Lese- und Schreibzugriffe threadsicher erfolgen, um Datenkorruption bei gleichzeitigen Zugriffen (Web-Request vs. Taster-Druck) zu vermeiden.

Systemintegration (Schnittstellen)

- **Hardware-Integration:** Der ESP32 steuert die Peripherie nicht direkt, sondern über eine definierte GPIO-Schnittstelle.
 - *Ausgänge:* PWM- oder Digitalsignale an die Gate-Treiber (MOSFETs) der Pumpen.
 - *Eingänge:* Digitale Inputs mit Pull-Up-Widerständen für die Taster.
- **Netzwerk-Integration:** Das System integriert sich als Client in ein bestehendes WLAN (Station Mode) oder fungiert als Access Point. Es werden keine externen APIs (Wetterdaten, Social Media) eingebunden.

Entwicklungsumgebung & Tools

- **Code-Editoren (IDEs):**
 - **Visual Studio Code:** Hauptentwicklungsumgebung für die React-Webapplikation und Code-Verwaltung.
 - **Visual Studio Code / Pymakr Plugin & Python Plugin:** Zum Flashen der Firmware, Hochladen der Dateien auf den ESP32 und zum Debugging (REPL-Konsole).
- **Versionskontrolle:**
 - **Git & GitHub:** Zur Versionierung des Quellcodes und zur Zusammenarbeit im Team. Es werden separate Repositories oder Ordner für Frontend und Firmware geführt.
- **Schaltplan & Design:**
 - **Fritzing:** Zur Erstellung der Schaltpläne und Verkabelungsskizzen.
 - **CAD-Software (CREO):** Für das Design des 3D-gedruckten Gehäuses und der Pumpenhalterungen

6.3 Problemanalyse

Fassen Sie die wichtigsten Probleme zusammen, die Sie erwarten. Wichtig ist vor allem, dass Sie für die wahrscheinlichsten Probleme bereits einen Lösungsansatz formulieren, um später Zeit zu sparen. Machen Sie sich auch über unwahrscheinliche Probleme Gedanken.



6.4 Qualität

Welche Anforderungen stellen Sie an die Qualität? Beschreiben Sie auch, wie die Qualitätssicherung, -kontrolle und -abnahme aussieht.

z.B.:

- **Testanforderungen:** Vorgaben zu Unit-Tests, Integrationstests, Lasttests und Benutzerakzeptanztests.
- **Fehlerbehandlung:** Beschreibung der Anforderungen an die Protokollierung und den Umgang mit Fehlern (z.B. Logging, Benutzerwarnungen).
- **Code-Standards:** Einhaltung von Coding Guidelines, Best Practices und Überprüfung des Codes durch Code Reviews.

7 Liefer- und Abnahmebedingungen

Hier wird festgehalten, in welchem Umfang und zu welchem Preis Sie an Ihren Kunden wann und wo liefern sollen.

Witerhin wird hier spezifiziert, wann das Projekt als abgeschlossen gilt und wer definiert, ob die Qualität stimmt. Es sollte klar festgelegt werden, wer für die Abnahme verantwortlich ist.

8 Anhang

Alle weiteren Dokumente oder Zahlen und Fakten, die als Hintergrund zu dem Projekt dienen.